

Laporan Tugas Besar Milestone 4

IF2224 Teori Bahasa Formal dan Otomata

Intermediate Code and Interpreter

Semester II Tahun 2025/2026

Kode Kelompok: MDC



Foto kelompok sebagai pengganti logo ITB

Disusun oleh:

13524041 - Nathan Adhika Santosa
13524053 - Muhammad Haris Putra Sulastianto
13524085 - Ariel Cornelius Sitorus
13524091 - Vara Azzara Ramli Pulukadang

Laboratorium Ilmu Rekayasa dan Komputasi
Sekolah Teknik Elektro dan Informatika
Program Studi Teknik Informatika
Institut Teknologi Bandung

Daftar Isi

1	Pendahuluan	3
2	Teori Singkat	3
2.1	Decorated AST dan Intermediate Code	3
2.2	Stack Machine	3
2.3	Activation Record	3
2.4	Instruksi Intermediate Code	4
2.5	Operasi OPR	5
3	Perancangan dan Implementasi	5
3.1	Bahasa dan Struktur Program	5
3.2	Fungsi dan Kelas Utama	5
3.3	Justifikasi Pilihan Implementasi	6
3.4	Alur Pipeline	7
3.5	Representasi Instruksi	7
3.6	Intermediate Code Generator	8
3.7	Bootstrap Jump	8
3.8	Translasi Control Flow	8
3.9	Procedure, Function, dan Recursion	9
3.10	Interpreter	9
3.11	Proteksi Runtime dan Bonus	9
4	Hasil Implementasi	10
4.1	Dokumentasi Hasil Implementasi	11
5	Pengujian	11
5.1	TC-01: Ekspresi Aritmatika dan Output	12
5.2	TC-02: For Loop dan If-Else	12
5.3	TC-03: Recursive Function	13
5.4	TC-04: Array dan Record	15
5.5	TC-05: Case Statement	16
5.6	TC-06: Bonus Runtime Protection	17
5.7	Analisis Pengujian	18
6	Kesimpulan dan Saran	18
6.1	Kesimpulan	18

6.2	Saran	19
A	Lampiran	21
A.1	Perintah Build dan Run	21
A.2	Ringkasan Testcase Tambahan	21
A.3	Link Release Repository GitHub	21
A.4	Diagram Pendukung	22
A.5	Pembagian Tugas	22

1 Pendahuluan

Milestone 4 menyelesaikan pipeline Arion sampai tahap eksekusi. Pada milestone sebelumnya, program sudah dapat ditokenisasi, diparse, dibentuk menjadi AST, lalu divalidasi secara semantik menjadi Decorated AST. Pada milestone ini, Decorated AST tersebut diterjemahkan menjadi Intermediate Code dan dijalankan oleh interpreter berbasis stack machine.

Dengan demikian, keluaran program tidak lagi hanya berupa token, parse tree, AST, Decorated AST, dan symbol table. Program juga menghasilkan daftar instruksi linear serta output runtime dari program Arion, terutama ketika program menggunakan `write`, `writeln`, atau `readln`.

2 Teori Singkat

2.1 Decorated AST dan Intermediate Code

Decorated AST adalah AST yang sudah diberi informasi semantik, seperti tipe data, indeks symbol table, lexical level, alamat variabel, indeks block, informasi array, dan informasi record. Informasi ini diperlukan oleh code generator karena generator tidak cukup hanya tahu bentuk sintaks, tetapi juga harus tahu lokasi memori dan tipe nilai yang sedang diterjemahkan.

Intermediate Code adalah bentuk instruksi linear yang lebih dekat dengan mesin virtual. Bentuk ini menggantikan traversal tree secara langsung. Ekspresi bersarang seperti `a + b * c` diratakan menjadi rangkaian LOD, LIT, dan OPR agar dapat dieksekusi satu per satu.

2.2 Stack Machine

Interpreter menggunakan model stack machine. Nilai literal, hasil ekspresi, alamat variabel, dan nilai sementara disimpan di stack. Instruksi mengambil operand dari stack, memprosesnya, lalu menaruh hasilnya kembali ke stack atau ke alamat memori tertentu.

Komponen utama interpreter adalah:

- `ip`, yaitu instruction pointer yang menunjuk instruksi aktif;
- `base`, yaitu awal activation record aktif;
- `top`, yaitu posisi teratas stack;
- `stack`, yaitu memori runtime untuk frame, variabel, parameter, dan nilai sementara.

2.3 Activation Record

Setiap procedure dan function memiliki activation record. Tiga slot awal frame selalu dipakai untuk data kontrol:

Offset	Isi
0	Static Link
1	Dynamic Link
2	Return Address
3 dan seterusnya	Parameter, variabel lokal, dan nilai return function

Konstanta `FRAME_OVERHEAD` bernilai 3. Oleh karena itu, alamat variabel dari symbol table ditambahkan dengan 3 saat diterjemahkan ke alamat runtime.

2.4 Instruksi Intermediate Code

Instruksi yang digunakan mengikuti instruksi inti stack machine dan beberapa instruksi tambahan untuk fitur runtime. Instruksi inti ditunjukkan pada Tabel 1.

Table 1: Instruksi inti intermediate code

Instruksi	Operasi	Keterangan
LIT	Load literal	Memasukkan literal integer, real, char, atau string ke stack.
LOD	Load value	Memuat nilai dari alamat variabel ke stack.
STO	Store value	Menyimpan nilai stack teratas ke alamat variabel.
CAL	Call	Memanggil procedure atau function.
INT	Initiate memory	Mengalokasikan frame runtime.
JMP	Jump	Melompat tanpa syarat ke alamat instruksi.
JPC	Conditional jump	Melompat jika kondisi bernilai false.
OPR	Operation	Menjalankan operasi aritmatika, relasional, logika, atau output.
RET	Return	Keluar dari procedure, function, atau program utama.

Instruksi tambahan pada implementasi ini digunakan untuk array, record, case, dan input.

Instruksi	Keterangan
LDA	Memasukkan alamat variabel ke stack.
LDI	Memuat nilai dari alamat tidak langsung.
STI	Menyimpan nilai ke alamat tidak langsung.
IDX	Menghitung offset array dan mengecek batas indeks.
DUP	Menduplikasi nilai teratas stack, dipakai pada <code>case</code> .
POP	Menghapus nilai teratas stack.
RED	Membaca input untuk <code>readln</code> .

2.5 Operasi OPR

Kode	Operasi	Keterangan
1	NEG	Negasi unary
2	ADD	Penjumlahan atau konkatenasi string
3	SUB	Pengurangan
4	MUL	Perkalian
5	DIV	Pembagian
6	MOD	Modulo
7	EQL	Sama dengan
8	NEQ	Tidak sama dengan
9	LSS	Kurang dari
10	GEQ	Lebih dari atau sama dengan
11	GTR	Lebih dari
12	LEQ	Kurang dari atau sama dengan
13	WRT	Output tanpa newline
14	WRTLN	Output dengan newline

3 Perancangan dan Implementasi

3.1 Bahasa dan Struktur Program

Program dikembangkan dengan C++17 dan tetap berada pada repositori yang sama dengan milestone sebelumnya. Komponen Milestone 4 ditambahkan tanpa mengganti pipeline lexer, parser, AST builder, dan semantic analyzer.

```
src/
|-- lexer.cpp / lexer.hpp
|-- parser.cpp / parser.hpp
|-- ast.cpp / ast.hpp
|-- semantic.cpp / semantic.hpp
|-- codegen.cpp / codegen.hpp
|-- interpreter.cpp / interpreter.hpp
'-- main.cpp
```

File penting Milestone 4 adalah:

- `codegen.hpp` dan `codegen.cpp`: representasi instruksi dan Intermediate Code Generator;
- `interpreter.hpp` dan `interpreter.cpp`: stack machine, activation record, dan runtime protection;
- `main.cpp`: integrasi pipeline sampai interpreter.

3.2 Fungsi dan Kelas Utama

Rancangan program dibuat modular agar setiap komponen memiliki tanggung jawab yang jelas. Ringkasan fungsi dan kelas utama ditunjukkan pada Tabel 2.

Table 2: Fungsi dan kelas utama Milestone 4

Komponen	Lokasi	Tanggung jawab
Instruction	codegen.hpp	Menyimpan satu instruksi Intermediate Code beserta operand, level, tipe nilai, literal string/real, dan metadata pemanggilan sub-program.
CodeGenerator	codegen.cpp	Melakukan traversal Decorated AST, mengeluarkan instruksi melalui <code>emit</code> , dan melakukan backpatching untuk jump.
generateCode	codegen.cpp	Entry point pembangkitan Intermediate Code dari Decorated AST dan symbol table.
Value	interpreter.hpp	Representasi nilai runtime agar integer, real, dan string dapat berada pada stack yang sama.
Interpreter	interpreter.cpp	Menjalankan siklus fetch-decode-execute, mengelola <code>ip</code> , <code>base</code> , stack frame, operasi OPR, CAL, RET, dan built-in I/O.
checkAddress dan validasi runtime	interpreter.cpp	Memastikan akses memori, jump target, indeks array, pembagian, dan operasi numerik tetap aman saat runtime.
main	main.cpp	Menghubungkan lexer, parser, AST builder, semantic analyzer, code generator, dan interpreter dalam satu pipeline eksekusi.

3.3 Justifikasi Pilihan Implementasi

Implementasi menggunakan Intermediate Code linear dan stack machine karena format ini sederhana untuk dieksekusi, mudah diuji, dan sesuai dengan kebutuhan milestone untuk menghasilkan Intermediate Code sekaligus output final program Arion. Model stack dipilih karena ekspresi, parameter, temporary value, dan activation record dapat direpresentasikan dengan mekanisme push/pop yang seragam.

Pemilihan struktur `Instruction` berbasis mnemonic, level, dan operand mempertahankan bentuk instruksi yang mudah dibaca. Metadata tambahan seperti `valueType`, `stringOperand`, `argCount`, dan `returnValue` dipakai untuk kebutuhan praktis bahasa Arion seperti string literal, real number, `readln`, serta procedure/function call tanpa mengubah format inti instruksi.

Array dan record diterjemahkan melalui address calculation agar assignment ke selector seperti `data[i]` dan `p.x` tidak diperlakukan sebagai kasus khusus di interpreter. Pendekatan ini membuat LDA, IDX, LDI, dan STI dapat dipakai ulang untuk load maupun store. Runtime protection ditempatkan di interpreter karena sebagian kondisi berbahaya,

seperti out-of-bounds index dan overflow numerik, baru diketahui dari nilai aktual saat program berjalan.

3.4 Alur Pipeline

Source Code Arion

```
-> Lexer
-> Parser
-> AST Builder
-> Semantic Analyzer
-> Decorated AST + Symbol Table
-> Intermediate Code Generator
-> Intermediate Code
-> Interpreter
-> Output runtime
```

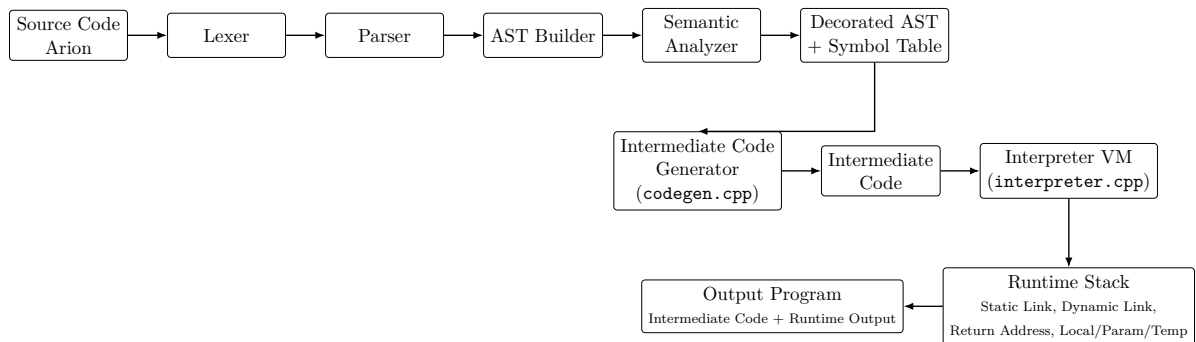


Figure 1: Diagram arsitektur pipeline Milestone 4 dari source code hingga output runtime.

Diagram pada Gambar 1 memperlihatkan pemisahan komponen front-end yang sudah dibangun pada milestone sebelumnya dan komponen back-end yang ditambahkan pada Milestone 4. Batas utama Milestone 4 berada pada **CodeGenerator** dan **Interpreter VM**; keduanya menerima Decorated AST dan symbol table, menghasilkan Intermediate Code, lalu menjalankan instruksi tersebut pada stack runtime.

3.5 Representasi Instruksi

Setiap instruksi disimpan sebagai struktur **Instruction**. Tiga field utama tetap berupa mnemonic, level, dan operand. Metadata tambahan dipakai oleh VM untuk string, real, **readln**, dan pemanggilan function/procedure.

```
struct Instruction {
    std::string mnemonic;
    int level = 0;
    int operand = 0;

    std::string valueType = "integer";
    double realOperand = 0.0;
    std::string stringOperand;

    int argCount = 0;
    bool returnsValue = false;
}
```

```
int returnOffset = 0;
};
```

Metadata tidak mengubah format dasar intermediate code. Metadata hanya membuat VM tetap dapat menangani tipe data dan call frame dengan benar.

3.6 Intermediate Code Generator

Generator bekerja dengan traversal Decorated AST. Entry point-nya adalah fungsi `generateCode`. Class internal `CodeGenerator` melakukan traversal node dan mengeluarkan instruksi melalui fungsi `emit`.

Pola utama generator adalah:

- literal diterjemahkan menjadi LIT;
- variabel tanpa selector diterjemahkan menjadi LOD;
- assignment sederhana diterjemahkan menjadi ekspresi lalu STO;
- array dan record diterjemahkan menjadi alamat dengan LDA, IDX, offset field, LDI, dan STI;
- operator biner diterjemahkan menjadi dua operand lalu OPR;
- control flow diterjemahkan dengan JPC, JMP, dan backpatching;
- procedure/function call diterjemahkan dengan CAL;
- `write`, `writeln`, dan `readln` diterjemahkan sebagai built-in runtime instruction.

3.7 Bootstrap Jump

Generator selalu mengeluarkan JMP di awal program. Instruksi ini melompati kode procedure/function declaration agar eksekusi langsung masuk ke main block. Body subprogram hanya dijalankan ketika dipanggil melalui CAL.

Contoh:

```
0 JMP 0 17    ; lompat ke main block
1 INT 0 5     ; awal function/procedure
...
16 RET
17 INT 0 5    ; awal main program
```

3.8 Translasi Control Flow

`if` dan `if-else` menggunakan JPC untuk keluar dari cabang ketika kondisi false. Jika terdapat `else`, generator menambahkan JMP agar cabang `then` tidak jatuh ke cabang `else`.

`while` menyimpan alamat awal kondisi, mengeluarkan JPC ke akhir loop, menjalankan body, lalu JMP kembali ke kondisi.

for diterjemahkan menjadi assignment nilai awal, pengecekan batas, body, increment/decrement, lalu **JMP** ke awal pengecekan. Arah **to** memakai **LEQ**, sedangkan **downto** memakai **GEQ**.

case memakai **DUP** agar nilai selector dapat dibandingkan ke beberapa label tanpa hilang. Setiap label dikompilasi menjadi perbandingan **EQL**, **JPC** ke label berikutnya, body, lalu **JMP** ke akhir case.

3.9 Procedure, Function, dan Recursion

Pemanggilan subprogram memakai **CAL**. Sebelum **CAL**, argumen dievaluasi ke stack. Instruksi **CAL** membawa metadata jumlah argumen, status return value, dan offset return value. Saat function selesai, **RET** mengambil nilai return dari activation record dan menaruhnya kembali ke stack caller.

Static link dan dynamic link membuat pemanggilan procedure/function tetap dapat mengakses variabel pada lexical level yang sesuai. Mekanisme ini juga memungkinkan recursive function seperti faktorial berjalan karena setiap call memiliki frame sendiri.

3.10 Interpreter

Interpreter menjalankan instruksi dengan loop fetch-decode-execute. Instruksi dibaca dari `code[ip]`, `ip` dinaikkan, lalu opcode dijalankan. Instruksi jump dapat mengubah `ip`; instruksi **CAL** membuat frame baru; instruksi **RET** memulihkan frame caller.

Nilai runtime disimpan dalam tipe **Value** dengan tiga jenis: integer, real, dan string. Dengan desain ini, operasi **write/writeln**, arithmetic real, dan string output dapat dijalankan oleh VM yang sama.

3.11 Proteksi Runtime dan Bonus

Interpreter tidak hanya mengeksekusi instruksi, tetapi juga memvalidasi kondisi berbahaya sebelum operasi dilakukan.

Proteksi	Implementasi
Stack overflow	Mengecek batas <code>MAX_STACK_SIZE</code> sebelum push dan alokasi frame.
Stack underflow	Mengecek jumlah nilai di atas batas frame sebelum pop.
Stack smashing/corruption	Melarang write ke static link, dynamic link, return address, dan alamat di luar frame.
Invalid jump target	Memvalidasi target <code>JMP</code> , <code>JPC</code> , dan <code>CAL</code> .
Out-of-bounds array access	Instruksi <code>IDX</code> mengecek indeks terhadap batas bawah dan atas array.
Division/modulo by zero	<code>DIV</code> dan <code>MOD</code> melempar runtime error jika pembagi nol.
Numerical overflow	Operasi integer memakai pengecekan batas <code>int</code> . Operasi real dicek dengan <code>isfinite</code> .
Invalid memory address	<code>checkAddress</code> memastikan alamat berada dalam frame yang dialokasikan.

4 Hasil Implementasi

Hasil akhir implementasi adalah pipeline yang dapat menerima source code Arion, membentuk intermediate code, dan menjalankannya. Fitur yang berhasil diimplementasikan diringkas pada Tabel 3.

Table 3: Ringkasan hasil implementasi

Fitur	Status
Intermediate Code Generator	Berjalan dari Decorated AST dan symbol table.
Stack Machine Interpreter	Menjalankan instruksi dengan <code>ip</code> , <code>base</code> , <code>top</code> , dan <code>stack</code> .
Ekspresi aritmatika dan relasional	Didukung melalui <code>OPR</code> .
<code>if</code> , <code>while</code> , <code>repeat</code> , <code>for</code>	Didukung dengan <code>JPC</code> dan <code>JMP</code> .
<code>case</code>	Didukung dengan <code>DUP</code> , <code>POP</code> , <code>EQL</code> , <code>JPC</code> , dan <code>JMP</code> .
Procedure/function call	Didukung dengan <code>CAL</code> , <code>RET</code> , parameter, dan return value.
Recursive function	Didukung melalui activation record terpisah per call.
Array dan record	Didukung dengan address calculation, <code>IDX</code> , <code>LDI</code> , dan <code>STI</code> .
String, real, <code>write/writeln</code> , <code>readln</code>	Didukung pada runtime VM.
Bonus proteksi runtime	Stack overflow/underflow, stack smashing, invalid jump, out-of-bounds, division by zero, modulo by zero, numerical overflow.

Contoh keluaran akhir untuk program sederhana:

```

=== INTERMEDIATE CODE ===
0 JMP 0 1
1 INT 0 5
2 LIT 0 5
3 STO 0 3
4 LOD 0 3
5 LIT 0 10
6 OPR 0 2
7 STO 0 4
8 LIT 0 0 ; string "Result = "
9 OPR 0 13
10 LOD 0 4
11 OPR 0 14
12 RET

INTERPRETER OUTPUT
Result = 15

```

4.1 Dokumentasi Hasil Implementasi

Hasil implementasi tidak hanya berupa tabel fitur, tetapi juga bukti keluaran program pada bagian pengujian. Setiap testcase memuat input program Arion, cuplikan Intermediate Code, output interpreter, dan gambar bukti terminal. Dokumentasi ini menunjukkan bahwa program memenuhi keluaran Milestone 4, yaitu Intermediate Code dan output sebenarnya ketika program menggunakan `write` atau `writeln`.

5 Pengujian

Pengujian laporan difokuskan pada enam testcase inti yang mewakili ekspresi, control flow, subprogram, struktur data, case, dan proteksi runtime. Setiap testcase menampilkan input program, cuplikan Intermediate Code, output interpreter, serta bukti screenshot terminal. Testcase lain tetap disimpan di direktori `test/milestone-4/` dan dipakai untuk verifikasi tambahan.

No	Testcase	Fokus	Hasil
TC-01	Ekspresi dan output	LIT, LOD, STO, OPR, string output	Berhasil
TC-02	Control flow	for, if-else, JPC, JMP	Berhasil
TC-03	Recursive function	CAL, RET, activation record, return value	Berhasil
TC-04	Array dan record	LDA, IDX, LDI, STI, field offset	Berhasil
TC-05	Case statement	Selector, branch comparison, DUP, POP	Berhasil
TC-06	Bonus runtime protection	Out-of-bounds dan numerical overflow	Berhasil

5.1 TC-01: Ekspresi Aritmatika dan Output

Tujuan. Menguji ekspresi aritmatika, assignment, string output, dan `writeln`.

```
program Hello;
var
  a, b : integer;
begin
  a := 5;
  b := a + 10;
  writeln('Result = ', b);
end.
```

Cuplikan hasil.

```
8 LIT 0 0 ; string "Result = "
9 OPR 0 13
10 LOD 0 4
11 OPR 0 14
12 RET
```

```
INTERPRETER OUTPUT
Result = 15
```

Hasil sesuai ekspektasi. String literal dicetak dengan `WRT`, nilai `b` dicetak dengan `WRTLN`, dan hasil arithmetic $5 + 10$ adalah 15.

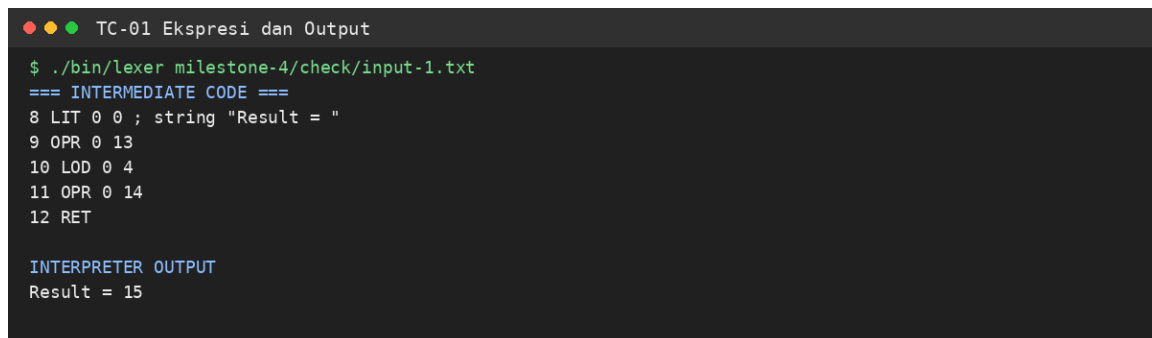


Figure 2: Bukti screenshot output TC-01

5.2 TC-02: For Loop dan If-Else

Tujuan. Menguji translasi loop dan percabangan menjadi instruksi linear.

```
program FlowRun;
var
  i, sum: integer;
begin
  sum := 0;
  for i := 1 to 5 do
  begin
    sum := sum + i;
  end;

  if sum = 15 then
    writeln(sum)
  else
```

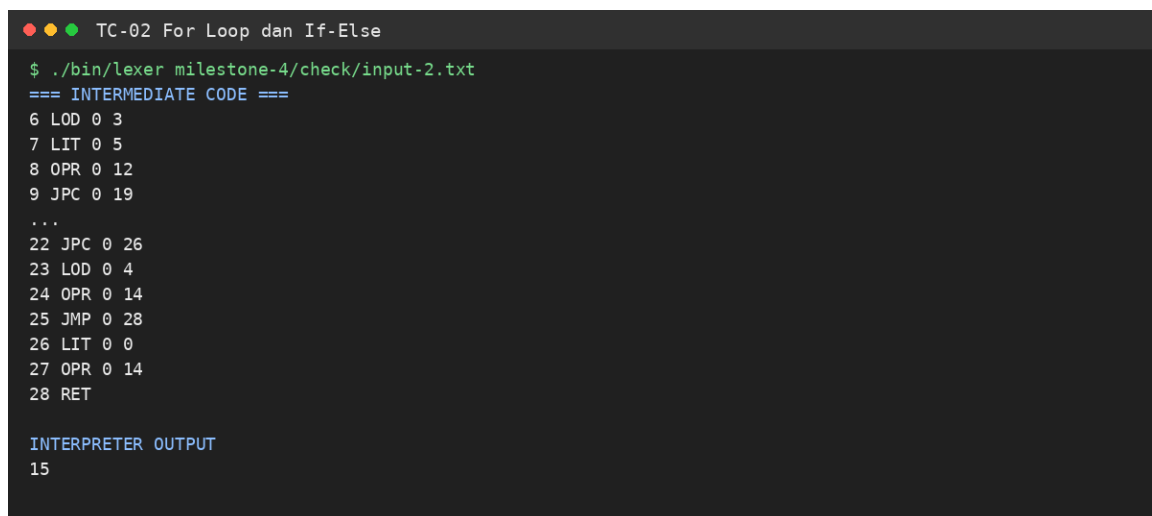
```
writeln(0);
end.
```

Cuplikan hasil.

```
6 LOD 0 3
7 LIT 0 5
8 OPR 0 12
9 JPC 0 19
...
19 LOD 0 4
20 LIT 0 15
21 OPR 0 7
22 JPC 0 26
23 LOD 0 4
24 OPR 0 14
25 JMP 0 28
26 LIT 0 0
27 OPR 0 14
28 RET
```

```
INTERPRETER OUTPUT
15
```

Hasil sesuai ekspektasi. Loop menghitung jumlah 1 sampai 5, lalu `if` memilih cabang benar karena `sum = 15`.



```
TC-02 For Loop dan If-Else
$ ./bin/lexer milestone-4/check/input-2.txt
=== INTERMEDIATE CODE ===
6 LOD 0 3
7 LIT 0 5
8 OPR 0 12
9 JPC 0 19
...
22 JPC 0 26
23 LOD 0 4
24 OPR 0 14
25 JMP 0 28
26 LIT 0 0
27 OPR 0 14
28 RET

INTERPRETER OUTPUT
15
```

Figure 3: Bukti screenshot output TC-02

5.3 TC-03: Recursive Function

Tujuan. Menguji function call, recursive call, parameter, return value, dan activation record.

```
program testrec;
var
  n, hasil : integer;

function faktorial(x : integer) : integer;
begin
```

```
if x <= 1 then
    faktorial := 1
else
    faktorial := x * faktorial(x - 1);
end;

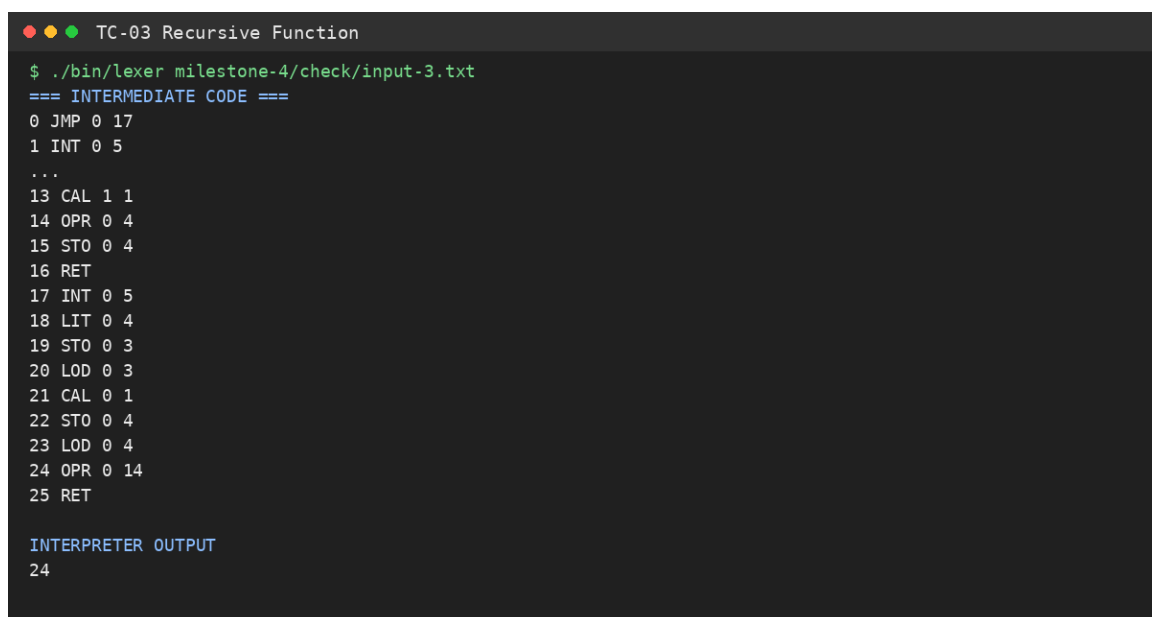
begin
    n := 4;
    hasil := faktorial(n);
    writeln(hasil);
end.
```

Cuplikan hasil.

```
0 JMP 0 17
1 INT 0 5
...
13 CAL 1 1
14 OPR 0 4
15 STO 0 4
16 RET
17 INT 0 5
18 LIT 0 4
19 STO 0 3
20 LOD 0 3
21 CAL 0 1
22 STO 0 4
23 LOD 0 4
24 OPR 0 14
25 RET
```

INTERPRETER OUTPUT
24

Hasil sesuai ekspektasi. Function `faktorial` dipanggil secara rekursif dan setiap call memiliki frame sendiri.



```
TC-03 Recursive Function
$ ./bin/lexer milestone-4/check/input-3.txt
=== INTERMEDIATE CODE ===
0 JMP 0 17
1 INT 0 5
...
13 CAL 1 1
14 OPR 0 4
15 STO 0 4
16 RET
17 INT 0 5
18 LIT 0 4
19 STO 0 3
20 LOD 0 3
21 CAL 0 1
22 STO 0 4
23 LOD 0 4
24 OPR 0 14
25 RET

INTERPRETER OUTPUT
24
```

Figure 4: Bukti screenshot output TC-03

5.4 TC-04: Array dan Record

Tujuan. Menguji address calculation untuk array dan record.

```
program MemoryRun;
type
  Point == record
    x: integer;
    y: integer;
  end;
var
  data : array [1..3] of integer;
  p : Point;
begin
  data[1] := 4;
  data[2] := 6;
  p.x := data[1] + data[2];
  writeln(p.x);
end.
```

Cuplikan hasil.

```
2 LDA 0 3
3 LIT 0 1
4 IDX 1 3 ; elsz 1
5 OPR 0 2
6 LIT 0 4
7 STI 0 0
...
17 LDA 0 3
18 LIT 0 1
19 IDX 1 3 ; elsz 1
20 OPR 0 2
21 LDI 0 0
22 LDA 0 3
23 LIT 0 2
24 IDX 1 3 ; elsz 1
25 OPR 0 2
26 LDI 0 0
27 OPR 0 2
28 STI 0 0
...
INTERPRETER OUTPUT
10
```

Hasil sesuai ekspektasi. Array memakai offset indeks, sedangkan record memakai offset field. Nilai $p.x$ menjadi $4 + 6 = 10$.

```

TC-04 Array dan Record
$ ./bin/lexer milestone-4/check/input-4.txt
=== INTERMEDIATE CODE ===
2 LDA 0 3
3 LIT 0 1
4 IDX 1 3 ; elsz 1
5 OPR 0 2
6 LIT 0 4
7 STI 0 0
...
27 OPR 0 2
28 STI 0 0

INTERPRETER OUTPUT
10

```

Figure 5: Bukti screenshot output TC-04

5.5 TC-05: Case Statement

Tujuan. Menguji translasi `case` menjadi rangkaian `comparison` dan `jump`.

```

program CaseTest;
var
  x: integer;
begin
  x := 2;
  case x of
    1: writeln(10);
    2: writeln(20);
    3: writeln(30);
  end;
end.

```

Cuplikan hasil.

```

4 LOD 0 3
5 DUP 0 0
6 LIT 0 1
7 OPR 0 7
8 JPC 0 13
...
13 DUP 0 0
14 LIT 0 2
15 OPR 0 7
16 JPC 0 21
17 POP 0 0
18 LIT 0 20
19 OPR 0 14
20 JMP 0 30
...
INTERPRETER OUTPUT
20

```

Hasil sesuai ekspektasi. Selector `x` bernilai 2 sehingga branch kedua dijalankan.

```

TC-05 Case Statement
$ ./bin/lexer milestone-4/check/input-5.txt
=== INTERMEDIATE CODE ===
4 LOD 0 3
5 DUP 0 0
6 LIT 0 1
7 OPR 0 7
8 JPC 0 13
...
13 DUP 0 0
14 LIT 0 2
15 OPR 0 7
16 JPC 0 21
17 POP 0 0
18 LIT 0 20
19 OPR 0 14
20 JMP 0 30

INTERPRETER OUTPUT
20

```

Figure 6: Bukti screenshot output TC-05

5.6 TC-06: Bonus Runtime Protection

Tujuan. Menguji proteksi runtime pada kondisi yang seharusnya tidak membuat interpreter crash.

TC-06A: Out-of-Bounds Array Access

```

program ArrayOobRun;
var
  data : array [1..5] of integer;
  i: integer;
begin
  i := 6;
  data[i] := 10;
end.

```

```
6 IDX 1 5 ; elsz 1
```

```
...
```

```
INTERPRETER OUTPUT
```

```
[RUNTIME ERROR] Runtime Error: Out-of-Bounds Array Access - index 6 not in [1..5]
```

TC-06B: Numerical Overflow

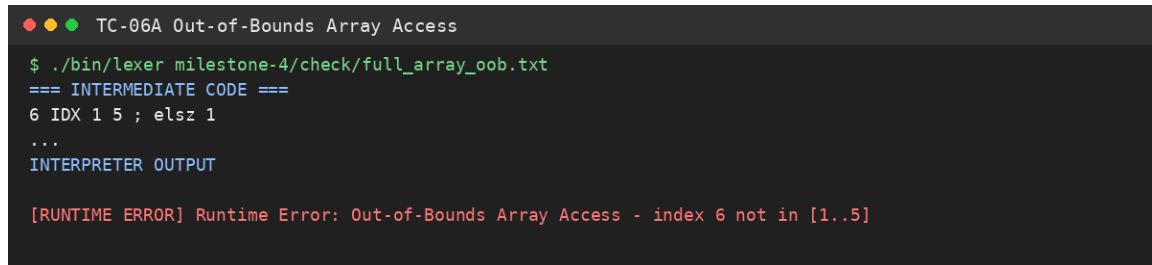
```

program OverflowRun;
var
  x: integer;
begin
  x := 2147483647 + 1;
  writeln(x);
end.

```

```
2 LIT 0 2147483647
3 LIT 0 1
4 OPR 0 2
...
INTERPRETER OUTPUT

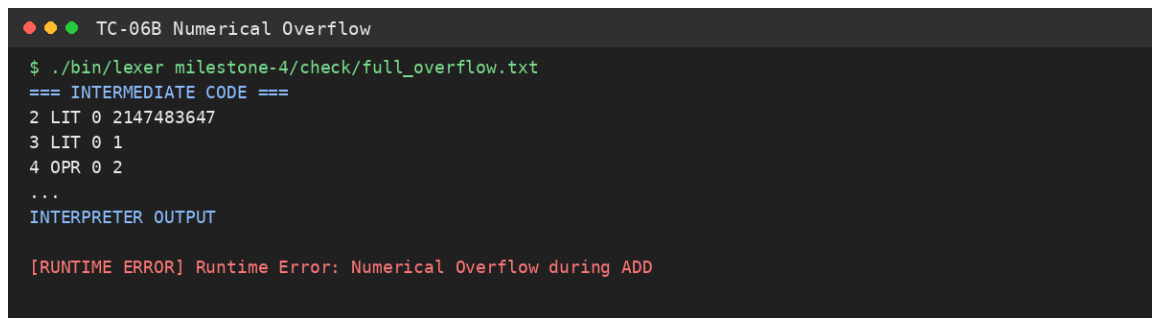
[RUNTIME ERROR] Runtime Error: Numerical Overflow during ADD
```

A terminal window titled "TC-06A Out-of-Bounds Array Access" showing the execution of a program. The output includes intermediate code and an interpreter output section. The error message is: "[RUNTIME ERROR] Runtime Error: Out-of-Bounds Array Access - index 6 not in [1..5]".

```
TC-06A Out-of-Bounds Array Access
$ ./bin/lexer milestone-4/check/full_array_oob.txt
=== INTERMEDIATE CODE ===
6 IDX 1 5 ; elsz 1
...
INTERPRETER OUTPUT

[RUNTIME ERROR] Runtime Error: Out-of-Bounds Array Access - index 6 not in [1..5]
```

Figure 7: Bukti screenshot output TC-06A

A terminal window titled "TC-06B Numerical Overflow" showing the execution of a program. The output includes intermediate code and an interpreter output section. The error message is: "[RUNTIME ERROR] Runtime Error: Numerical Overflow during ADD".

```
TC-06B Numerical Overflow
$ ./bin/lexer milestone-4/check/full_overflow.txt
=== INTERMEDIATE CODE ===
2 LIT 0 2147483647
3 LIT 0 1
4 OPR 0 2
...
INTERPRETER OUTPUT

[RUNTIME ERROR] Runtime Error: Numerical Overflow during ADD
```

Figure 8: Bukti screenshot output TC-06B

Kedua subcase berhasil. Interpreter menghentikan eksekusi dengan pesan runtime error yang jelas.

5.7 Analisis Pengujian

Seluruh testcase inti menghasilkan keluaran sesuai ekspektasi. Pengujian membuktikan bahwa generator dapat menangani ekspresi, control flow, subprogram, recursive call, array, record, case, dan built-in procedure. Bonus runtime protection juga aktif karena error berbahaya dicegat di level interpreter, bukan dibiarkan mengakses memori secara bebas.

6 Kesimpulan dan Saran

6.1 Kesimpulan

Milestone 4 berhasil mengintegrasikan Intermediate Code Generator dan Interpreter ke pipeline Arion. Decorated AST dari semantic analyzer dapat diterjemahkan menjadi instruksi linear seperti LIT, LOD, STO, INT, JMP, JPC, CAL, RET, dan OPR. Interpreter berbasis

stack machine dapat menjalankan instruksi tersebut sampai menghasilkan output runtime.

Implementasi juga mendukung fitur lanjutan seperti procedure, function, recursion, array, record, case, real, string, `readln`, serta proteksi runtime untuk stack, jump target, address, indeks array, division by zero, modulo by zero, dan numerical overflow.

6.2 Saran

Pengembangan selanjutnya dapat difokuskan pada optimasi intermediate code, visualisasi stack untuk debugging, dan pemisahan format intermediate code agar dapat disimpan serta dibaca ulang sebagai file mandiri.

Referensi

- [1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. Pearson, 2006.
- [2] Wirth, N. *Compiler Construction*. Addison-Wesley, 1996.
- [3] Tim Asisten IF2224. *Spesifikasi Milestone 4 - Tubes IF2224 TBFO*. Institut Teknologi Bandung, 2026.
- [4] Repository kelompok MDC - Mediocre. *MDC-Tubes-IF2224-2026*. 2026.

A Lampiran

A.1 Perintah Build dan Run

```
make

./bin/lexer milestone-4/check/full_array_oob.txt

./bin/lexer milestone-4/check/full_case_nomatch.txt

./bin/lexer milestone-4/check/full_for.txt

./bin/lexer milestone-4/check/full_downto.txt

./bin/lexer milestone-4/check/full_case.txt

./bin/lexer milestone-4/check/full_array.txt

./bin/lexer milestone-4/check/full_record.txt

./bin/lexer milestone-4/check/full_real.txt

./bin/lexer milestone-4/check/full_overflow.txt

./bin/lexer milestone-4/check/full_stack_overflow.txt

echo 40 | ./bin/lexer milestone-4/check/full_readln.txt
```

A.2 Ringkasan Testcase Tambahan

File	Output utama
full_array_oob.txt	[RUNTIME ERROR] Runtime Error: Out-of-Bounds Array Access - index 6 not in [1..5]
full_case_nomatch.txt	99
full_for.txt	15
full_downto.txt	15
full_case.txt	20
full_array.txt	10
full_record.txt	15
full_real.txt	3.75
full_overflow.txt	[RUNTIME ERROR] Runtime Error: Numerical Overflow during ADD
full_stack_overflow.txt	[RUNTIME ERROR] Runtime Error: Stack Overflow
full_readln.txt	input 40, output 42

A.3 Link Release Repository GitHub

<https://github.com/mhps-null/MDC-Tubes-IF2224-2026/releases/tag/v0.4.1>

Tag release mengikuti format `v0.4.1`, yaitu milestone 4 pengumpulan pertama. Jika dilakukan revisi, tag berikutnya menggunakan `v0.4.2` dan seterusnya.

A.4 Diagram Pendukung

Diagram arsitektur Milestone 4 sudah dicantumkan pada Gambar 1. File gambar diagram dapat disimpan sebagai berkas pendukung pada folder `doc/` bersama laporan PDF.

A.5 Pembagian Tugas

Nama	Kontribusi	Persentase
Nathan Adhika Santosa	P2: Ekstensi ICG untuk control flow, jump, dan pemanggilan subprogram.	22%
Muhammad Haris Putra Sulastianto	P4: ALU, proteksi memori, testing, dan laporan.	34%
Ariel Cornelius Sitorus	P1: Fondasi ICG untuk deklarasi, alokasi memori, dan ekspresi aritmatika.	22%
Vara Azzara Ramli Pulukadang	P3: Core VM, stack machine, activation record, dan proteksi stack/alur.	22%

Table 4: Pembagian tugas kelompok